DevOps Challenge: KIWA UNO

Infraestrutura e Deploy

C. Trancoso

Visão geral

Aplicação web estática (HTML/CSS/JS) servida via Nginx.

API desenvolvida em Node.js (Express) com base de dados PostgreSQL.

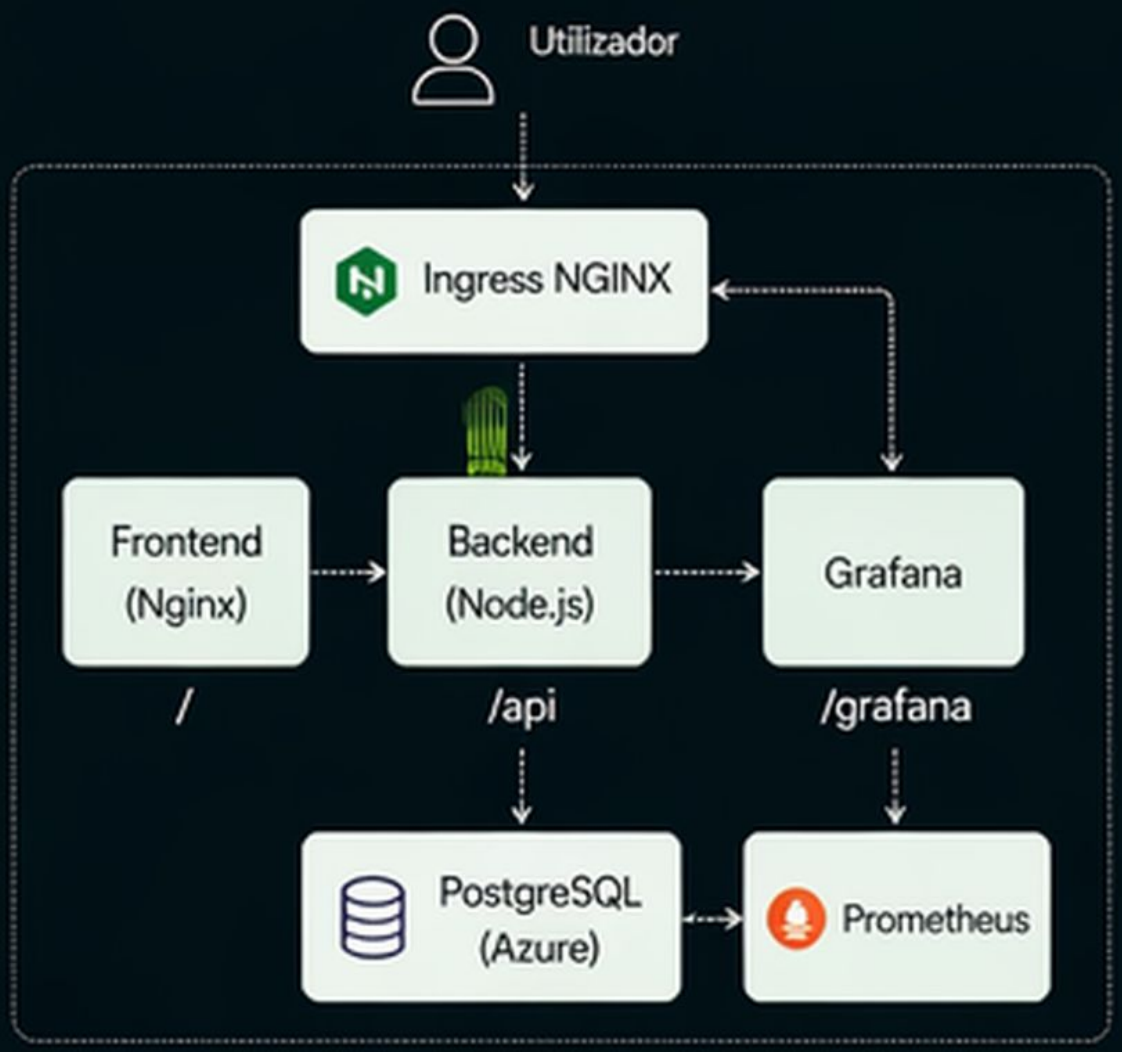
Infraestrutura na Microsoft Azure utilizando AKS, ACR e PostgreSQL.

Automação completa com Terraform, GitHub Actions e Kubernetes.

Monitorização com Prometheus e Grafana para visibilidade total.

Arquitetura

- Frontend estático servido via Nginx
- Backend (Node.js) exposto via /api
- PostgreSQL gerido no Azure
- Ingress NGINX para roteamento
- Monitorização com Prometheus + Grafana



IaC com Terraform

Resource Group

AKS
(Kubernetes)

ACR
(Container Registry)

PostgreSQL
(Azure)

Monitorização
(Prometheus + Grafana)

Infraestrutura como código

- Reprodutível
- Versionada
- Modular

Ambientes isolados

- Dev
- Prod

Segurança

- Network
- RBAC
- Secrets via GitHub e Key Vault (futuro)

Alta disponibilidade

- AKS gerido
- PostgreSQL flexível (HA e backups)

CI/CD e Deploy

Push de código	CI - Build & Test	Push para ACR	CD - Deploy	Aplicação
GitHub Repository	• Instala dependências • Executa testes • Build das imagens	Frontend Backend	Deploy no AKS via kubectl	Disponivel via Ingress

Ambientes: dev e prod | Aprovação manual para produção

Monitorização e Observabilidade

Prometheus

Coleta métricas de aplicações e cluster

Grafana

Dashboards e alertas para métricas e logs

Logs, Uptime e Operação

Verificar Uptime

curl http://20.67.219.180/api/health

Resposta esperada:

{ "status": "ok" }

Ver Logs

kubectl logs -l app=backend

kubectl logs -f deployment/backend

kubectl logs -l app-frontend

Eventos do Kubernetes

kubectl get events

kubectl describe pod <pod>

URLs principais

http://20.67.219.180/	http://20.67.219.180/api/health	http://20.67.219.180/grafana
Frontend	Healthcheck da API	Dashboards Grafana

Ambientes: Dev vs Prod

Dev

- Ambiente de desenvolvrento
- Deploy automático
- Menos restrições
- Testes e validações
- Recursos otimizados para custo

Prod

- Ambiente de produção
- Aprovação manual (GitHub Environments)
- Alta disponibilidade
- Monitorização e alertas
- Backup e segurança reforçada

Mesma base de código e infrasstrutura como código (Terraform)

Tecnologias Utilizadas

Stack completa da solução

1. Cloud & Infraestrutura

- Microsoft Azure
- Azure Kubernetes Service (AKS)
- Azure Container Registry (ACR)
- Azure Database for PostgreSQL

2. Infraestrutura como Código (IaC)

- **Terraform**
  IaC com Terraform

3. Backend

- Node.js
- Express.js

4. Frontend

- HTML5
- CSS3
- JavaScript (Vanilla)
- NGINX (Web Server)

5. Containers & Orquestração

- Docker
- Kubernetes
- NGINX Ingress Controller

6. CI/CD

- GitHub
- GitHub Actions

7. Banco de Dados

- **PostgreSQL**
  (Azure Database for PostgreSQL)

8. Monitorização & Observabilidade

- Prometheus (Métricas & Alertas)
- Grafana (Dashboards)

9. Testes

- Jest (Testes Unitários)